NAQLA

LAB 1

Lab 1 — Scaffold Shahna

Agentic Backend Engineering

Agentic Backend Engineering — Build Scalable Backends With and For AI Agents



Lab 1 — Scaffold Shahna

Overview

Goal

By the end of today you will have a running Laravel 12 application for **Shahna Logistics**, a Libyan freight company, with its complete domain model in PostgreSQL: cities, distances, carriers, vehicles, quotes, shipments and shipment events. You will have built it *with* Claude Code, and you will have checked every line the agent produced.

Today is about two habits that you will use for the rest of the week:

1. **Explore → Plan → Implement → Verify.** The agent never writes code before you have read and approved a plan.
2. **You are the engineer.** The agent is a fast pair who is sometimes confidently wrong. Your job is to catch that.

What you will build

Piece	Where it lives
A domain plan written with the agent	`docs/plans/day1-domain.md`
Migrations for 7 tables	`database/migrations/`
Eloquent models with relationships and casts	`app/Models/`
Backed enums for status, vehicle type, vehicle status	`app/Enums/`
`ShipmentStatus::canTransitionTo()` with a Pest unit test	`app/Enums/ShipmentStatus.php`, `tests/Unit/`
Factories for every model	`database/factories/`
A seeder with 8 Libyan cities, distances, carriers and vehicles	`database/seeders/`

Starting point

```
git clone https://github.com/OthmanHaba/shahna-api.git
cd shahna-api
git checkout day1-start
git switch -c my-day1
```

The `day1-start` tag contains a fresh Laravel 12 app with Sail (PostgreSQL 16 + Redis 7), Pest, Sanctum and Laravel Boost installed, a `CLAUDE.md` with project conventions, `docs/requirements.md` (the Shahna product brief) and an empty `docs/plans/` folder.

Time budget

Part	Step	Minutes
Part 1 (11:00–12:30)	1. Boot the app with Sail	20
	2. Connect Claude Code and Boost	10
	3. Explore the codebase with the agent	15
	4. Read the brief and write the plan	30
	Checkpoint 1	15
Part 2 (14:15–15:45)	5. Enums, by hand	20
	6. `canTransitionTo` test-first	15
	7. Migrations and models with the agent	25
	8. Factories and seeder	20
	Checkpoint 2 and commit	10

> **TIP**
> If you fall behind, do not panic and do not skip verification to "catch up". At any time you can jump to the reference solution with `git checkout day1-end`. Finishing fewer steps properly is worth more than finishing all of them blindly.

Part 1 — Boot, Explore, Plan (11:00–12:30)

Step 1 — Boot the app with Sail (20 min)

What and why. Laravel Sail runs PHP, PostgreSQL and Redis in Docker, so you do not need PHP installed on your laptop. Every command in this course runs *inside* the Sail container. This step is done **by hand**, without the agent: you must understand how your environment starts before you let anything else touch it.

> **WARNING**
> **By hand.** Do not ask Claude Code to do this step. If your environment is broken, the agent will "fix" it in ways you cannot explain later.

Install the PHP dependencies with a throwaway Composer container (you only need this once, because `vendor/` is not committed):

```
docker run --rm \
  -u "$(id -u):$(id -g)" \
  -v "$(pwd):/var/www/html" \
  -w /var/www/html \
  laravelsail/php84-composer:latest \
  composer install --ignore-platform-reqs
```

Create your environment file and add a shell alias so you can type `sail` instead of `./vendor/bin/sail`:

```
cp .env.example .env
alias sail='sh $([ -f sail ] && echo sail || echo vendor/bin/sail)'
```

Add the alias line to your `~/.zshrc` or `~/.bashrc` so it survives a new terminal.

Start the containers, generate the app key, and run the migrations:

```
sail up -d
sail artisan key:generate
sail artisan migrate
```

Check that everything is up:

```
sail ps
```

Expected output (columns trimmed):

```
NAME                    SERVICE    STATUS
shahna-api-laravel.test-1   laravel.test   Up
shahna-api-pgsql-1          pgsql          Up (healthy)
shahna-api-redis-1          redis          Up (healthy)
```

Open `http://localhost` in a browser: you should see the Laravel welcome page. Then run the test suite:

```
sail artisan test
```

```
   PASS  Tests\Unit\ExampleTest
  ✓ that true is true

   PASS  Tests\Feature\ExampleTest
  ✓ the application returns a successful response

  Tests:    2 passed (2 assertions)
```

Check your `.env`. Open it and confirm these lines. You should be able to explain each one.

```
DB_CONNECTION=pgsql
DB_HOST=pgsql
DB_PORT=5432
DB_DATABASE=shahna
REDIS_HOST=redis
```

`DB_HOST=pgsql` is not a typo: inside Docker, containers reach each other by service name, not `localhost`.

> **WARNING**
>
> **Common failures.** Port 80 already in use: set `APP_PORT=8080` in `.env` and use `http://localhost:8080`. Port 5432 in use (a local PostgreSQL): set `FORWARD_DB_PORT=54320`. "Permission denied" on `vendor/`: you skipped the `-u "$(id -u):$(id -g)"` flag; delete `vendor/` and rerun.

Step 2 — Connect Claude Code and Laravel Boost (10 min)

What and why. Laravel Boost is an MCP server that gives the agent *real* information about your app: installed package versions, database schema, routes, logs, and version-specific Laravel documentation. Without it, the agent guesses from training data, which may describe Laravel 9 or a package API that never existed.

Install Boost's Claude Code integration for this project. The installer asks which agents you use: choose Claude Code. It writes `.mcp.json` (which starts the MCP server with `artisan boost:mcp` through Sail) and appends Boost's guidelines to `CLAUDE.md`:

```
sail artisan boost:install
```

Review the `CLAUDE.md` diff with `git diff CLAUDE.md` before you commit it.

Start Claude Code in the project root:

```
claude
```

Inside the session, type `/mcp`. You should see `laravel-boost` listed as **connected**. If it shows **failed**, check that `sail ps` shows the containers running, then restart Claude Code.

Now read `CLAUDE.md` yourself (in your editor, not via the agent). This file is loaded into the agent's context on every session. It tells the agent the project conventions: Laravel 12, Pest, PostgreSQL, money as `decimal(10,2)`, run commands through `sail`, small diffs, tests with every change.

> **TIP**
>
> Every token in `CLAUDE.md` is paid for on every request. Keep it short and specific. "Write clean code" is useless; "Money columns are `decimal(10,2)`, never float" changes behaviour.

Step 3 — Explore the codebase with the agent (15 min)

What and why. The first phase of the loop is **Explore**. Before asking for any change, have the agent read the project and report back. This fills its context with facts about *this* codebase instead of generic Laravel knowledge, and it lets you check whether the agent actually understands what is there.

AGENT PROMPT

Agent prompt

```
Do not change any files. Explore this repository and give me a short report:
1. Laravel version, PHP version, and the versions of Pest, Sanctum and Boost
   (use the Boost tools, not guesses).
2. Which database and cache are configured, and where that is set.
3. What tables exist right now (use the Boost database schema tool).
4. What routes exist (use the Boost route list tool).
5. A summary of docs/requirements.md in 5 bullet points.
6. Anything in CLAUDE.md I should know before we write code.
Cite the file path for every claim.
```

What to check in the agent's output.

- Good: it calls Boost tools (you will see tool calls such as `application-info`, `database-schema`, `list-routes` in the transcript) and reports Laravel **12.x**.
- Good: every claim has a file path you can open and confirm.
- Red flag: it says "Laravel 11" or "MySQL" without a source. Ask: "Where did you read that?"
- Red flag: it edits a file. You said "do not change any files". Press `Esc` to interrupt, then reject the change.

Open two of the cited files yourself and confirm the claims are true.

Step 4 — Read the brief and write the domain plan (30 min)

What and why. This is the **Plan** phase. A written plan is cheap to change; a migration that has already shipped is not. You will produce `docs/plans/day1-domain.md`, a short document describing each table, its columns, types, constraints and relationships.

First, **by hand**, spend 5 minutes reading `docs/requirements.md` and write down on paper three questions you have about the domain (for example: “Can a shipment exist without a quote?”).

Now switch Claude Code into **plan mode** (press `Shift+Tab` until the status line shows *plan mode*). In plan mode the agent can read but not edit.

AGENT PROMPT

Agent prompt

```
Read docs/requirements.md and CLAUDE.md. We are designing the Day 1 domain
model only. Write a plan for docs/plans/day1-domain.md covering these models,
and nothing else:

City (name_en, name_ar, code — one of TIP, BEN, MRA, SEB, ZAW, TOB, SRT, GHA)
CityDistance (from_city_id, to_city_id, km)
Carrier (name, phone)
Vehicle (carrier_id, plate, type enum van|box_truck|flatbed|reefer,
         capacity_kg, status enum available|busy|maintenance, current_city_id)
Quote (user_id, origin_city_id, destination_city_id, weight_kg, vehicle_type,
       price_lyd decimal(10,2), expires_at)
Shipment (reference "SHN-" + 6 uppercase alphanumerics, user_id,
          quote_id unique, vehicle_id nullable, origin_city_id,
          destination_city_id, weight_kg, vehicle_type, price_lyd decimal(10,2),
          status, pickup_at)
ShipmentEvent (shipment_id, status, note, occurred_at)

Also add a role column to users: enum customer|dispatcher|admin, default customer.

For every table list: columns with PostgreSQL types, nullability, foreign keys
with on-delete behaviour, unique constraints, and indexes, with one line of
justification each. List the enums as PHP backed enums in App\Enums.
List the order the migrations must run in.
Do NOT add tables, columns, packages or features that are not listed here.
End with a list of open questions for me.
```

What to check in the plan.

Check	Good	Red flag
Money	`price_lyd numeric(10,2)`	`float`, `double`, or integer without explanation
Shipment reference	`string`, unique, indexed	Auto-increment ID used as the public reference
`quote_id` on shipments	unique foreign key	Not unique (one quote could be booked twice)
City codes	unique `char(3)` or short string	No uniqueness

Check	Good	Red flag
`CityDistance`	unique on `(from_city_id, to_city_id)`	Duplicates possible
Scope	Exactly the listed models	Invented `Payment`, `Invoice`, `Driver`, `Address` tables
Enums	`ShipmentStatus`, `VehicleType`, `VehicleStatus`, `UserRole`	Plain strings with no enum

Compare the agent's open questions with the three you wrote by hand. Answer them in the chat (examples: "Distances are stored in both directions", "Deleting a carrier is restricted if it has vehicles", "weight_kg is an unsigned integer").

When you are happy, leave plan mode (`Shift+Tab`) and ask:

AGENT PROMPT

Agent prompt

```
Write the agreed plan to docs/plans/day1-domain.md, including my answers
to the open questions. Do not create any other files.
```

Open the file and read it top to bottom. Edit anything you disagree with **by hand**. This document is now the contract for the rest of the day.

```
git add docs/plans/day1-domain.md
git commit -m "docs: day 1 domain plan"
```

CHECKPOINT

Checkpoint 1 (12:30)

`sail ps` shows `laravel.test`, `pgsql` and `redis` running.

`/mcp` in Claude Code shows `laravel-boost` connected.

`docs/plans/day1-domain.md` exists, is committed, and lists exactly seven Shahna tables plus the `role` column on `users`.

`sail artisan test` still shows **2 passed** (you have not changed code yet).

`curl -s http://localhost/up` returns HTTP 200 (Laravel's built-in health route):

```
curl -s -o /dev/null -w "%{http_code}\n" http://localhost/up
```

```
200
```

Part 2 — Enums, Models, Data (14:15–15:45)

Step 5 — Write the enums by hand (20 min)

What and why. PHP backed enums give you a closed set of values that the database, the code and the tests all agree on. You will write these yourself so you understand exactly what the agent is building on top of.

> **WARNING**
> **By hand.** Write all four enums yourself. Use the agent only to *review* them afterwards.

Create `app/Enums/ShipmentStatus.php`:

```php
<?php

namespace App\Enums;

enum ShipmentStatus: string
{
    case Pending = 'pending';
    case Confirmed = 'confirmed';
    case Assigned = 'assigned';
    case PickedUp = 'picked_up';
    case InTransit = 'in_transit';
    case Delivered = 'delivered';
    case Cancelled = 'cancelled';

    public function canTransitionTo(self $next): bool
    {
        // You will implement this in Step 6, test-first.
        return false;
    }
}
```

Now write, in the same style:

- `app/Enums/VehicleType.php` with `van`, `box_truck`, `flatbed`, `reefer`.
- `app/Enums/VehicleStatus.php` with `available`, `busy`, `maintenance`.
- `app/Enums/UserRole.php` with `customer`, `dispatcher`, `admin`.

Check that PHP can parse them:

```bash
sail artisan tinker --execute="dump(App\Enums\VehicleType::cases());"
```

You should see four `App\Enums\VehicleType` cases printed.

Step 6 — `canTransitionTo` test-first (15 min)

What and why. The shipment lifecycle is the most important rule in Shahna: `pending → confirmed → assigned → picked_up → in_transit → delivered`, and `cancelled` is allowed only from `pending`, `confirmed` or `assigned`. A rule like this deserves a test that lists every allowed and forbidden move. You will have the agent write the **test first**, run it red, then implement.

🤖 **AGENT PROMPT**

Agent prompt

```
First show me a plan, then wait for my approval.

Write a Pest unit test at tests/Unit/ShipmentStatusTest.php for
App\Enums\ShipmentStatus::canTransitionTo(). Rules:
- Allowed: pending→confirmed, confirmed→assigned, assigned→picked_up,
  picked_up→in_transit, in_transit→delivered.
- Allowed: pending→cancelled, confirmed→cancelled, assigned→cancelled.
- Everything else is forbidden, including staying in the same status,
  skipping steps, going backwards, and any move out of delivered or cancelled.
Use Pest datasets so every allowed pair and a representative set of forbidden
pairs are listed explicitly. Do NOT change ShipmentStatus.php yet.
Then run `sail artisan test --filter=ShipmentStatus` and show me it fails.
```

What to check.

- Good: a dataset with the 8 allowed pairs, and a dataset of forbidden pairs that includes `delivered → cancelled`, `picked_up → cancelled`, `pending → assigned` (skip) and `assigned → pending` (backwards).
- Good: the test fails for the allowed pairs (your stub returns `false`). A test that passes against a stub that always returns `false` is not testing the allowed path.
- Red flag: the agent "helpfully" also implements `canTransitionTo`. Reject that change; you asked for red first.

Run it yourself:

```
sail artisan test --filter=ShipmentStatus
```

```
  FAILED  Tests\Unit\ShipmentStatusTest > it allows valid transitions with ...
  Tests:    8 failed, 12 passed
```

(The exact numbers depend on how many forbidden pairs are in the dataset; the forbidden ones pass because the stub returns `false`.)

Now implement it. Try it **by hand** first for 5 minutes using a `match` expression. If you get stuck, ask the agent:

AGENT PROMPT

Agent prompt

```
Implement ShipmentStatus::canTransitionTo() so that the tests in
tests/Unit/ShipmentStatusTest.php pass. Use a single match expression on
$this. Do not modify the test file. Run the test and show the result.
```

A correct implementation looks like this:

```php
public function canTransitionTo(self $next): bool
{
    return match ($this) {
        self::Pending => in_array($next, [self::Confirmed, self::Cancelled], true),
        self::Confirmed => in_array($next, [self::Assigned, self::Cancelled], true),
        self::Assigned => in_array($next, [self::PickedUp, self::Cancelled], true),
        self::PickedUp => $next === self::InTransit,
        self::InTransit => $next === self::Delivered,
        self::Delivered, self::Cancelled => false,
    };
}
```

```
sail artisan test --filter=ShipmentStatus
```

```
  Tests:    20 passed
```

WARNING

Red flag: if the agent edits the test to make it pass (removing a forbidden pair, flipping an expectation), that is the most common way agents "cheat". Always run `git diff tests/` after an implementation step.

Commit:

```
git add app/Enums tests/Unit/ShipmentStatusTest.php
git commit -m "feat: shipment status enum with transition rules"
```

Step 7 — Migrations and models with the agent (25 min)

What and why. Now the agent does the bulk work, guided by the plan you approved. Ask for **small diffs**: migrations first, reviewed, then models.

AGENT PROMPT

Agent prompt

```
Follow docs/plans/day1-domain.md exactly. First show me the list of files you
will create, then wait for approval.

Step A only: create the migrations, in dependency order, using
`sail artisan make:migration`. Requirements:
- foreignId()->constrained() for every foreign key, with the on-delete
  behaviour from the plan.
- decimal('price_lyd', 10, 2) for money. No floats.
- unique index on shipments.quote_id, shipments.reference, cities.code,
  and (from_city_id, to_city_id) on city_distances.
- A migration that adds `role` to users with default 'customer'.
- Enum columns use $table->enum() with the values taken from the PHP enum,
  e.g. array_column(VehicleType::cases(), 'value'). Never hard-code the list.
Do not create models yet. When done, run `sail artisan migrate:fresh`
and show me the output.
```

Review the migrations yourself. Open every new file under `database/migrations/`. This is the step where the course "Pitfalls" bite hardest: an unreviewed migration becomes permanent schema.

- Good: timestamps in order so `cities` runs before `city_distances`, `carriers` before `vehicles`, `quotes` before `shipments`.
- Good: `down()` methods that drop exactly what `up()` created.
- Red flag: `$table->float('price_lyd')` or `$table->double(...)`.
- Red flag: `$table->enum(...)` with a hard-coded list of values instead of `array_column(SomeEnum::cases(), 'value')`; the PHP enum and the column will drift apart.
- Red flag: extra tables or columns not in the plan.
- Check: the on-delete behaviour of `shipments.user_id` must match your plan. The reference solution uses `cascadeOnDelete()` (deleting a user deletes their shipments); `restrict` keeps shipment history safer in production. Either is acceptable if your plan says so and you can explain it.

Inspect the real schema, independent of the agent's summary:

```
sail psql -c '\d shipments'
```

```
      Column        |              Type              | Nullable
--------------------+--------------------------------+----------
 id                 | bigint                         | not null
 reference          | character varying(10)          | not null
 user_id            | bigint                         | not null
 quote_id           | bigint                         | not null
 vehicle_id         | bigint                         |
 ...
 price_lyd          | numeric(10,2)                  | not null
Indexes:
    "shipments_pkey" PRIMARY KEY, btree (id)
    "shipments_quote_id_unique" UNIQUE CONSTRAINT, btree (quote_id)
    "shipments_reference_unique" UNIQUE CONSTRAINT, btree (reference)
```

Now the models:

AGENT PROMPT

Agent prompt

```
Step B: create Eloquent models City, CityDistance, Carrier, Vehicle, Quote,
Shipment, ShipmentEvent following docs/plans/day1-domain.md.
- Use the Laravel 12 `casts()` method. Cast enum columns to the enums in
  App\Enums, price_lyd to 'decimal:2', datetimes to 'datetime'.
- Add belongsTo / hasMany relationships described in the plan, with return
  types. Add User→role cast and User→shipments() and User→quotes().
- Use $fillable explicitly, never $guarded = [].
- Shipment: generate `reference` in a `creating` model event if empty, as
  "SHN-" + 6 uppercase alphanumerics (Str::upper(Str::random(6))).
Small diff, no other files. Then run `sail artisan test`.
```

What to check.

- Good: `protected function casts(): array` (Laravel 11+ style), not the old `protected $casts` property. Both work, but your `CLAUDE.md` asks for the new style.
- Good: `public function vehicle(): BelongsTo` with imports from `Illuminate\Database\Eloquent\Relations`.
- Red flag: `$guarded = []`. This disables mass-assignment protection.
- Red flag: relationship names that do not match the columns (for example `origin()` without specifying `'origin_city_id'` as the foreign key, which makes Eloquent look for `origin_id`).

Prove the relationships work in Tinker. This is a **by hand** check:

```
sail artisan tinker
```

```
> $c = App\Models\City::create(['name_en' => 'Tripoli', 'name_ar' => 'طرابلس', 'code' =>
      'TIP']);
> $c->code;
= "TIP"
> App\Enums\ShipmentStatus::from('pending')
      ->canTransitionTo(App\Enums\ShipmentStatus::Delivered);
= false
```

Commit:

```
git add database/migrations app/Models
git commit -m "feat: shahna domain migrations and models"
```

Step 8 — Factories and seeder (20 min)

What and why. Factories make tests readable (`Shipment::factory()->create()`); the seeder gives everyone the same realistic data for the API work tomorrow.

AGENT PROMPT

Agent prompt

```
Plan first, then wait for approval.

1. Create factories for every Shahna model. Factories must create valid
   related records (e.g. ShipmentFactory creates its own Quote, User, cities).
   Add factory states: UserFactory dispatcher()/admin(), and
   ShipmentFactory status(ShipmentStatus $status).
2. Create CitySeeder with exactly these 8 cities (code, name_en, name_ar):
   TIP Tripoli طرابلس, BEN Benghazi بنغازي, MRA Misrata مصراتة,
   SEB Sebha سبها, ZAW Zawiya الزاوية, TOB Tobruk طبرق, SRT Sirte سرت,
   GHA Ghat غات.
   In the same seeder, add road distances in km for all 28 city pairs, stored
   in both directions (same value). Put the distances in one array constant
   so I can review them.
3. Create FleetSeeder with 4 carriers and 2 vehicles of every type in every
   city (capacity_kg: van 1500, box_truck 8000, flatbed 20000, reefer 10000).
4. In DatabaseSeeder, call CitySeeder and FleetSeeder, then create three demo
   users with password "password": customer@shahna.test (customer),
   dispatcher@shahna.test (dispatcher), admin@shahna.test (admin).
5. Seeders must be idempotent (updateOrCreate / firstOrCreate).
Run `sail artisan migrate:fresh --seed` and `sail artisan test`.
```

Verify the distances by hand. LLMs are bad at geography numbers. Pick three pairs and sanity-check them against a map (approximate road distances: Tripoli–Misrata about 210 km, Tripoli–Benghazi about 1,000 km, Tripoli–Sebha about 780 km). If a value is wildly off, fix it **by hand** in the seeder array. Anything within about 10% is fine for this course.

What to check.

- Good: 8 cities, 56 distance rows (8 × 7 ordered pairs).

- Good: running `CitySeeder` and `FleetSeeder` twice does not create duplicates. (The demo users in `DatabaseSeeder` use `create()`, so a full `db:seed` on a seeded database fails on the unique email; use `migrate:fresh --seed` to reseed everything.)
- Red flag: the agent seeds a "Libya" country table, addresses, or drivers. Remove anything not in the plan.
- Red flag: Arabic names that look garbled. Check the file is UTF-8.

Run the counts yourself:

```
sail artisan migrate:fresh --seed
sail artisan tinker --execute="dump(App\Models\City::count(), App\Models\CityDistance::count(),
        App\Models\Vehicle::count());"
```

```
8
56
64
```

(64 = 8 cities × 4 vehicle types × 2 vehicles.)

Run the city and fleet seeders a second time and confirm the counts did not change:

```
sail artisan db:seed --class=CitySeeder
sail artisan db:seed --class=FleetSeeder
sail artisan tinker --execute="dump(App\Models\City::count(), App\Models\CityDistance::count(),
        App\Models\Vehicle::count());"
```

Finally, ask the agent to review its own day, then judge the review yourself:

AGENT PROMPT

Agent prompt

```
Review the diff of branch my-day1 against day1-start (git diff day1-start).
Compare it with docs/plans/day1-domain.md. List: (1) anything in the code
that is not in the plan, (2) anything in the plan that is missing in the code,
(3) any money stored as float, any missing unique constraint, any missing
foreign key. Do not fix anything; just report.
```

CHECKPOINT

Checkpoint 2 (15:45)

```
sail artisan migrate:fresh --seed
sail artisan test
```

```
   PASS  Tests\Unit\ShipmentStatusTest
  ✓ it allows valid transitions with ...
  ✓ it forbids invalid transitions with ...
  ...
  Tests:    23 passed
```

The reference solution (`day1-end`) has **23** tests. Your exact count depends on your dataset size; what matters is **zero failures**.

Quick data check through the app (there is no API yet; that is tomorrow):

```
sail artisan tinker --execute="echo App\Models\City::where('code','BEN')->first()
      ->toJson(JSON_UNESCAPED_UNICODE);"
```

```
{"id":2,"name_en":"Benghazi","name_ar":"بنغازي","code":"BEN","created_at":"...","updated
       _at":"..."}
```

Commit and you are done:

```
git add -A
git commit -m "feat: factories and seeders"
```

If you're stuck

The reference solution for today is the `day1-end` tag. Compare your work with it rather than copying it blindly:

```
# What files differ between your branch and the reference?
git diff --stat day1-end

# Look at one file in the reference without changing your branch
git show day1-end:app/Enums/ShipmentStatus.php

# Diff one file
git diff day1-end -- database/seeders/CitySeeder.php
```

If your branch is beyond repair, start fresh from the reference (your work stays on `my-day1`):

```
git stash
git switch -c my-day1-reset day1-end
sail artisan migrate:fresh --seed
sail artisan test
```

Tomorrow everyone starts from `day1-end`, so nobody is left behind.

Stretch goals

1. **Database check constraints.** Add a migration with `DB::statement` that enforces `weight_kg > 0` and `price_lyd ⩾ 0` on `quotes` and `shipments`, and `from_city_id <> to_city_id` on `city_distances`. Write a Pest test proving the database rejects a negative weight.
2. **Model-level guard.** Add `Shipment::transitionTo(ShipmentStatus $next, ?string $note = null)` that throws a domain exception when `canTransitionTo` is false, and otherwise updates the status and records a `ShipmentEvent` in one database transaction. Test-first.
3. **Arabic labels.** Add a `label(string $locale)` method to each enum that returns an English or Arabic label. Ask the agent to write it, then check every Arabic string with a native speaker in the room.
4. **Write a custom slash command.** Create `.claude/commands/review-migration.md` with a reusable prompt that reviews migrations against the plan. Use it on your own migrations.

Reflection

Answer these in two or three sentences each and bring them to tomorrow's standup.

1. Name one thing the agent got wrong or added without being asked today. How did you notice it, and what would have happened if you had not?
2. Which checks did you do *outside* the agent (Tinker, `psql`, `git diff`, a map)? Which of them caught something the agent's own summary did not mention?
3. Look at your `CLAUDE.md`. What one line would you add to it, based on today, so the agent makes fewer mistakes tomorrow?